Lagoon Epibenthos YOLO (lagoon-epibenthos-yolo)

python 3.14.3

YOLO v11

License MIT

Welcome to the official code repository for the paper:
"Implementing Optimized Computer Vision Algorithm To Underwater Imagery For Identification and Spatial Analysis Of Epibenthic Fauna In Shallow Lagoon Waters"

About the repository

This repository contains the supplementary code, datasets, and trained model used to detect and spatially analyse epibenthic fauna (*Paranemonia* sp., *Anemonia* sp., and *Brachyura* sp.) in the turbid, shallow waters of Logarou Lagoon, Greece, using USV-collected underwater imagery and a custom-trained YOLOv11s object detector.

The custom-trained model weights (`best.pt`) and all exported training metric files are provided. Ready-to-use extracted data (`.xlsx`) files are also included so readers can reproduce the statistical analyses without running the full detection pipeline.

Repository structure

```
lagoon-epibenthos-yolo-supplementary/
│
├── models/
│   └── best.pt                              # Trained YOLOv11s weights (for
the 3 classes)
│   ├── yolo11s.pt                           # The original pre-trained small
YOLOv11 model
│   └── selected-model-metrics/              # Metric outputs for the selected
model used in the study
│
├── data-files/
│   ├── Habitat_Epibenthos_statistics.xlsx   # Pre-extracted detection &
habitat data
│   └──Epibenthos finder YOLOv11 most fit optimization.xlsx #The search list
for the most
│                                                            #appropriate custom-
trained model
│
├── imagery-files/                          # images unedited in `/raw/`
directory and already
│                                           # enhanced in the `/enhanced/`
directory to be used
│                                           # with inference scripts
```

```
│
├── scripts/
│   │
│   ├── CLAHE_underwater_preprocessor_github_v2.py   # Stage 1: image
enhancement
│   ├── unified_statistical_analysis_github_v2.py    # Stage 2: statistical
analysis
│   ├── habitat_parameters.txt                       # Needed file for the
stage 2 - statistical analysis
│   │
│   ├── For-inference/
│   │   └── YOLO_on_pics_to_table_and_annot.py       # Run detection on new
images
│   │
│   ├── For-training/                                 # Reproduce model
training
│   │   ├── config_imbalanced.yaml
│   │   ├── yolo_trainer_for_imbalanced.py
│   │   ├── dataset_Kfold_splitter-trainer.py
│   │   └── YOLO_optimizer_supplementary.py
│   │
│   └── aux-scripts/                                  # Dataset preparation
utilities
│       ├── dataset_augmentor.py
│       └── per_class_validator.py
│
├── requirements.txt
└── README.md
```

⚙️ Prerequisites

To run these scripts you will need Python and a code editor. We recommend **Visual Studio Code (VS Code)**.

1. Install Python

1. Download **Python 3.14.3** (or newer) from the official Python website.
2. Run the installer.
3. **Critical:** check the **"Add Python to PATH"** box before clicking Install.

2. Install Visual Studio Code

1. Download VS Code from code.visualstudio.com.
2. Install with default settings.
3. Go to the Extensions tab (`Ctrl+Shift+X`) and install the **Python** extension by Microsoft.

🛠️ Installation & Setup

Step 1 — Clone or download the repository

```
git clone https://github.com/yourusername/lagoon-epibenthos-yolo.git
cd lagoon-epibenthos-yolo
```

Alternatively, download the `.zip` from GitHub and extract it.

Step 2 — Open in VS Code
Click `File > Open Folder...` and select the repository folder. Open a terminal via `Terminal > New Terminal`.

Step 3 — Create a virtual environment (recommended)

```
# Windows
python -m venv venv
.\venv\Scripts\activate

# macOS / Linux
python3 -m venv venv
source venv/bin/activate
```

Step 4 — Install dependencies

```
pip install -r requirements.txt
```

🚀 Running the pipeline

1. Image enhancement

`scripts/CLAHE_underwater_preprocessor_github_v2.py`

Applies a three-stage radiometric correction pipeline to raw USV-collected underwater images: adaptive grey-world colour-cast correction → CLAHE on the CIE L* channel → optional per-channel histogram stretching. All EXIF/GPS metadata is preserved in the output files. The script is fully interactive — run it and answer the prompts for input/output directories and processing parameters.

```
python scripts/CLAHE_underwater_preprocessor_github_v2.py
```

2. Statistical analysis

`scripts/unified_statistical_analysis_github_v2.py`

Reads the pre-extracted detection and habitat-area data from `data-files/Habitat_Epibenthos_statistics.xlsx` and produces the core statistical results reported in the paper: chi-square goodness-of-fit habitat-selectivity tests (observed vs. expected counts with Bonferroni-corrected standardised residuals), Kruskal–Wallis tests with post-hoc pairwise comparisons, and Spearman rank correlations between species abundances and habitat variables. The script is interactive — run it and follow the console menu.

```
python scripts/unified_statistical_analysis_github_v2.py
```

3. YOLO inference on new images (optional)

`scripts/For-inference/YOLO_on_pics_to_table_and_annot.py`

Runs the trained YOLOv11s model (`models/best.pt`) on a folder of (preferably enhanced) images. For every image it saves a JPEG-compressed annotated copy with bounding boxes drawn, and at the end writes a single CSV table containing per-image detection counts and mean confidence scores for each of the three classes. Edit the three path constants at the bottom of the script before running.

```
python scripts/For-inference/YOLO_on_pics_to_table_and_annot.py
```

🔧 Training scripts (For-training/)

These scripts are provided for full reproducibility. They are **not required** to validate the paper's results — the trained weights and extracted data are sufficient for that. GPU access is required for practical training times.

Script	Purpose
`config_imbalanced.yaml`	Dataset configuration file: paths to train/val/test splits, class names, and inverse-frequency class weights (1× *Paranemonia*, 15× *Anemonia*, 20× *Brachyura*). Edit the `path` field to match your local dataset location.
`yolo_trainer_for_imbalanced.py`	Main training script. Loads `yolo11s.pt` and fine-tunes it on the annotated dataset using the hyperparameters reported in the paper (AdamW, cosine LR schedule, multi-scale training, mosaic/mixup augmentation). Exports the best checkpoint to ONNX on completion.

Script	Purpose
`dataset_Kfold_splitter-trainer.py`	Creates stratified 5-fold cross-validation splits from the augmented dataset and optionally trains a separate model on each fold, reporting per-fold and aggregated mAP50 / mAP50-95 / precision / recall with 95% confidence intervals. Used to verify generalisation before final single-model training.
`YOLO_optimizer_supplementary.py`	Bayesian hyperparameter search (60 iterations, 50 epochs each) over dropout, geometric augmentation strengths, and flip probabilities, using the fixed learning-rate and colour-augmentation values established in a prior manual search. Produces the `best_hyperparameters.yaml` used by `yolo_trainer_for_imbalanced.py`.

Auxiliary scripts (aux-scripts/)

Script	Purpose
`dataset_augmentor.py`	Offline dataset augmentation using the Albumentations library. Applies colour jitter, affine transforms, Gaussian noise/blur, and horizontal flips. Common classes (*Paranemonia* and *Anemonia*) are augmented ×3; rare classes (*Brachyura*) ×5, to partially correct class imbalance before training.
`per_class_validator.py`	Evaluates the trained model on the held-out test split and produces a detailed per-class performance report (precision, recall, mAP50, mAP50-95, instance counts, and inference speed), saved as both `.txt` and `.csv`.

Contact & Citation

If you use this code or methodology in your research, please cite our paper (citation details pending publication). For code-related issues, please open an Issue on GitHub.